

COMP2041 Week 1

Nicholas Sebastian

**Thank you Kyu-Sang Kim for the
foundation of these slides!**

Overview

- How will the tutorials work?
- Tutorial Goals
- Tips for succeeding in COMP2041
- Lab Goals
- Assignments
- Weekly Tests
- Course webpage run-through
- Glory of Regex
- Grep
- Tutorial Questions

How Will The Tutorials Work

In-person tutorials on Thursday 11AM-2PM

11AM - 12PM: Tutorial

12PM - 2PM: Lab

No tutorials in Week 6 (Flex Week)

Tutorial Goals

- **Note: Tutorials are not a replacement for the lectures and other material (they go into much more depth on the theory than here)**
- **1 hour per week means I will focus primarily on concepts that have been known to be hard or confusing**
- Tutorials reinforce what you already learnt at the lectures
- However, I will still try and briefly explain every 'new' concept as I go through the tutorial questions. (As it is the first week, I will be a lot more lenient and explain more than not)
- If something is missing, let me know so I can plan a recap next week.

Tutorial Tips to Succeed in COMP2041/9044

- I **strongly** encourage you to **ask questions**
 - **No question is dumb or silly** especially if it will help you understand content
 - It is infinitely better to ask questions **at the time of confusion** rather than later
 - Put your hand up whenever, I promise I won't be offended :)
 - It may be scary to ask or answer questions which is completely normal but it makes classes MUCH more helpful and enjoyable if you try your best to do so. I can promise that it gets easier with practice
- Before class, attempt some tutorial questions so that the tutorial isn't the first time facing them.

TL;DR DO NOT FEEL AFRAID TO STOP ME IF YOU ARE CONFUSED

Tutorial Tips to Succeed in COMP2041/9044 (cont.)

- **I strongly** encourage you to **use all of the resources that COMP2041 provides you**, even the bare minimum (labs and tutorials)
 - Nowadays, students feel tutorials and labs aren't too important when it comes to attendance. - **VERY WRONG**
 - Smartest people I know attended their non-compulsory classes until their final term.
- **Do not use AI for your submissions.**
 - You will get caught. And if you somehow don't get caught, the exam hurdle will catch you.
 - Last term's exam was the first I've seen to have exam marks be scaled up.
 - You will be much more satisfied with your course experience if you don't just AI every assignment given to you.
 - You can use AI to aid your learning.

Lab Goals

- You should expect to complete all standard lab exercises
- **Labs are marked and are very beneficial to assignments**
- **You can still get an almost perfect score without doing any challenge exercises but I highly recommend you try to attempt them as some are much easier than others**

Assignments

- If you're doing this course early into your degree, the length of the assignments may surprise you.
- Start early, use all the resources at your disposal (labs, help sessions, forum)
- Always watch the lecture intro to the assignment / assignment video before.
 - This will save you immense amount of time when reading the assignment specification (typically intentionally vague)

Weekly Tests

Will go into detail more when they are officially out (Week 3)

Tl;dr - weekly hour long online test with three coding questions related to prior week's content.

Course webpage run-through

COMP(2041|9044) - 26T2 [Course Outline](#) [Course Timetable](#) [Course Forum](#)

Software Construction

Course Resources

Administrivia: [Course Outline](#) | [Course Timetable](#)

[COMP2041 Handbook](#) | [COMP9044 Handbook](#)

Shell: [Shell Style Guide \(For POSIX Shell\)](#)

Python: [Python Documentation \(For Python3.13\)](#) | [Python Style Guide \(PEP 8\)](#)

Team: [The 2026 T2 COMP2041/9044 Tutor Team](#)

Platforms: [Lecture Recordings \(via YouTube\)](#)

[Online Tut-Labs & Help Sessions \(via BbCollaborate \(via Moodle\)\)](#)

[Course Forum](#)

Assessment: [Autotests](#), [Submissions](#), [Marks](#)

[Give online: submission](#) | [Give online: sturec](#)

Course Content Week-by-Week

Week 1

Tutorial [Questions](#)

Laboratory [Exercises](#)

Monday Week 1 Lecture Topics [Course Intro](#)
[Filters](#)

Announcements

- **Welcome to Week 1 of COMP2041**
- Any important announcements will be said here every week but do check the course website just in case

The Glory of Regex

What are Regular Expressions (regex)

- A regular expression is a pattern string comprising of a combination of “**operators**” or “**syntax**” that describes a set of strings within text
- These regex strings can then be used to **locate and manage text** which will be a core part of Week 1 lab exercises
- You will most likely use regular expressions in any software-related job at some point

Example of Operators and Syntax

Quantifiers	
<code>a b</code>	Match either "a" or "b"
<code>?</code>	Match either "a" or "b"
<code>+</code>	One or more
<code>*</code>	Zero or more
<code>*?</code>	Zero or more, but stop after first match
<code>{N}</code>	Exactly N number of times (Where N is number)
<code>{N, M}</code>	From N to M number of times (Where N and M are numbers)

Pattern Collections	
<code>[A-Z]</code>	Match any uppercase character from "A" to "Z"
<code>[a-z]</code>	Match any lowercase character from "a" to "z"
<code>[0-9]</code>	Match any number
<code>[asdf]</code>	Match any character that's either "a", "s", "d", or "f"
<code>[^asdf]</code>	Match any character that's not any of the following: "a", "s", "d", or "f"

General Tokens	
<code>.</code>	Any character
<code>\n</code>	Newline character
<code>\t</code>	Tab character
<code>\s</code>	Any whitespace character (Including \t, \n, etc)
<code>\S</code>	Any non-whitespace character
<code>\w</code>	Any word character (Upper/lowercase letters, 0-9, _)
<code>\W</code>	Any non-word character
<code>\b</code>	Word boundary (Matches between characters)
<code>\B</code>	Non-word boundary
<code>^</code>	The start of a line
<code>\$</code>	The end of a line
<code>\\</code>	The literal character "\"

Flags	
<code>g</code>	Global, match more than once
<code>m</code>	Force \$ and ^ to match each newline individually
<code>i</code>	Make the regex case-insensitive

Groups	
<code>(...)</code>	Capture group (Matches any 3 characters)
<code>(?: ...)</code>	Non-capture group (Matches any 3 characters)
<code>(?<name> ...)</code>	Named capture group Group is called "name"

Named Back Reference	
<code>\k<name></code>	Reference named capture group "name" in query

Lookahead and Lookbehind	
--------------------------	--

grep tool

- In unix-based systems, *grep* is a tool that allows you to print lines that match a given regex pattern
- *grep* in most distributions will understand 3 very similar but different regex pattern syntax options:
 - Basic Regular Expressions (BRE) - default
 - Extended Regular Expressions (ERE) - most useful IMO
 - Perl-compatible Regular Expressions (PCRE)
 - Not so useful with the removal of Perl from COMP2041
- `$ man 1 grep` (for more usage information)

Tutorial Questions

To The Lab!